FATEC POMPEIA

P2

RELATÓRIO CONSOLIDADO
ARQUITETURA E ATIVIDADES DO 2º BIMESTRE

Avaliação individual · 2026.2

Joice Soato Brito

Produto: TestFlow AI
https://github.com/joicesoato/testflow-ai
https://joice-brito-isw055.lapps.studio/

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução
2 Metodologia
3 Arquitetura do SaaS
3.1 Visão geral e componentes
3.2 Onde roda
3.3 Fluxo de uma requisição
3.4 Fluxo de assinatura
3.5 Modelo de dados do domínio
3.6 Segurança
3.7 Observabilidade, CI/CD e documentação da API
3.8 Camadas IaaS, PaaS e SaaS do projeto
4 Quadro de entregas
5 Atividades realizadas
5.1 Atividade 7 — Plano Premium — Stripe
5.2 Atividade 8 — SaaS, PaaS, IaaS e documentação
5.3 Atividade 9 — Fork e plano de negócio
5.4 Atividade 10 — Publicação do produto
5.5 Atividade 11 — SaaS completo
6 Considerações finais
7 Declaração de autoria

1 Introdução

O presente relatório apresenta o desenvolvimento individual do TestFlow AI no segundo bimestre da disciplina ISW055 — Introdução à Computação em Nuvem, do curso de Tecnologia em Sistemas Inteligentes da FATEC Pompeia, semestre 2026.2. O trabalho teve como objetivo consolidar os conhecimentos de arquitetura em nuvem, conteinerização, integração de serviços, segurança, implantação e disponibilização de uma solução no modelo Software as a Service (SaaS).

O projeto partiu da base técnica desenvolvida no primeiro bimestre, inicialmente estruturada como um catálogo de filmes. Na atividade de fork e plano de negócio, essa base foi reaproveitada e direcionada a um novo problema: organizar e apoiar a automação de testes de APIs REST. O produto resultante, denominado TestFlow AI, permite que desenvolvedores, profissionais de QA, freelancers e pequenas equipes organizem projetos, cadastrem cenários, gerem casos com auxílio de inteligência artificial, executem verificações e consultem o histórico dos resultados.

A proposta responde à dificuldade de manter cenários, requisições e resultados de testes dispersos entre ferramentas, scripts e planilhas. A plataforma reúne essas etapas em um único ambiente e acrescenta autenticação, isolamento de dados por usuário, auditoria e planos Free e Pro com cobrança recorrente integrada ao Stripe. Este documento descreve a metodologia, a arquitetura implantada, as decisões de segurança e as atividades 7 a 11, incluindo evidências que serão complementadas com capturas de tela do ambiente.

2 Metodologia

O desenvolvimento foi conduzido individualmente, com evolução incremental do repositório GitHub e validações no ambiente de desenvolvimento (GitHub Codespaces), na infraestrutura conteinerizada gerenciada pelo Portainer e nos serviços externos utilizados pelo produto. As principais ferramentas foram Git e GitHub para versionamento e rastreabilidade; Docker e Docker Compose para empacotamento; Docker Hub para distribuição das imagens; Portainer para implantação; MariaDB para persistência; Redis Streams para eventos de auditoria; MinIO como serviço de objetos provisionado; Groq para geração assistida de casos; e Stripe em modo de teste para o fluxo de assinatura.

As evidências e datas foram confrontadas com o histórico de commits, as issues de entrega e o pull request do repositório. Para as atividades 9 e 10, foram consideradas as issues de entrega dirigidas ao professor; para a atividade 11, o pull request de implementação do fluxo SaaS e os commits de correção e integração. Os horários abaixo estão apresentados no horário de Brasília (UTC−3), convertidos quando o GitHub registrou o horário em UTC. As capturas de tela solicitadas pelo modelo do professor permanecem indicadas em espaços próprios para inserção posterior.

A existência de um recurso no código foi diferenciada de sua comprovação visual em produção. Por isso, funcionalidades não integradas ou não comprovadas foram identificadas como limitações, sem atribuir ao produto resultados que não estejam sustentados pelo repositório ou pelas validações realizadas. As capturas de tela das entregas e dos resultados foram inseridas nas respectivas seções deste relatório, complementando os registros de código, histórico, issues e pull requests.

3 Arquitetura do SaaS

3.1 Visão geral e componentes

O TestFlow AI utiliza uma aplicação web principal integrada a dois microsserviços internos e a serviços de suporte. O navegador acessa a aplicação pelo domínio HTTPS; a aplicação Express concentra as rotas de negócio e a comunicação com o MariaDB. O auth-service valida tokens e permissões; o log-service recebe eventos de auditoria e os encaminha ao Redis Streams. A aplicação também se integra à API da Groq para geração assistida e ao Stripe para Checkout, gerenciamento da assinatura e recebimento de webhooks.

Os serviços app, auth-service, log-service, Redis e MinIO estão definidos na composição de produção do Portainer e compartilham a rede Docker catalogo_net. O MariaDB é externo à Stack e acessado por variáveis de ambiente. O Stripe e a Groq são serviços externos, acessados pelo backend. O MinIO está provisionado, mas o upload e a recuperação de arquivos de evidência ainda não foram integrados ao fluxo funcional do produto.

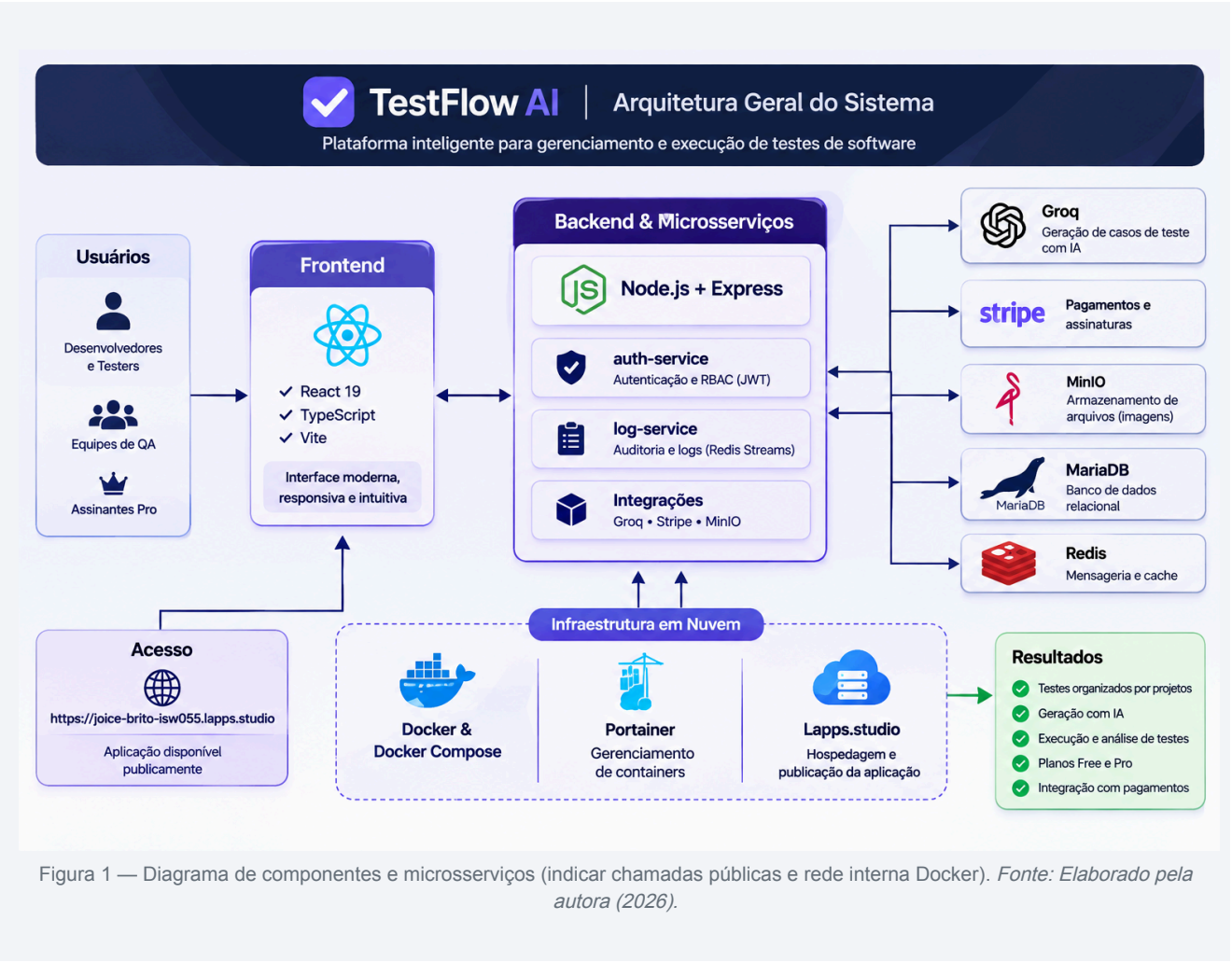


Figura 1 — Diagrama de componentes e microsserviços (indicar chamadas públicas e rede interna Docker). *Fonte: Elaborado pela autora (2026).*

Componente	Responsabilidade	Tecnologia / imagem	Público?
app	Interface, API de negócio e regras do SaaS	React 19 / Node.js 22 / Express 5; joicesoato/testflow-ai	Sim — domínio HTTPS
auth-service	Cadastro, login, JWT e autorização RBAC	Node.js; joicesoato/testflow-ai-auth	Não — rede Docker
log-service	Recepção e consulta de eventos de auditoria	Node.js; joicesoato/testflow-ai-log	Não — rede Docker
Redis	Streams para eventos de auditoria	Redis 7 Alpine	Não — rede Docker
MariaDB	Usuários e dados do domínio/assinatura	MariaDB 11 externo à Stack	Não — conexão privada configurada
MinIO	Serviço de objetos provisionado	MinIO	Não — rede Docker; upload não integrado
Stripe	Checkout, assinatura, Billing Portal e webhooks	Serviço SaaS externo	Webhook HTTPS → app
Groq	Geração assistida de casos de teste	API externa	Backend → API externa

3.2 Onde roda

A aplicação é publicada na infraestrutura da disciplina por meio de uma Stack Docker gerenciada pelo Portainer, com imagens mantidas no Docker Hub. O repositório de origem do produto é joicesoato/testflow-ai, na branch main. O domínio público informado no README e na issue de implantação é https://joice-brito-isw055.lapps.studio/. A aplicação interna escuta na porta 3000 e o Compose publica a porta 8211 do host para essa porta.

A Stack contém os serviços app (joicesoato/testflow-ai), auth-service (joicesoato/testflow-ai-auth), log-service (joicesoato/testflow-ai-log), Redis 7 e MinIO. Redis e MinIO utilizam volumes persistentes. Os serviços possuem health checks configurados para a aplicação, autenticação, logs e Redis. O MariaDB não é iniciado por essa Stack: sua conexão é fornecida externamente por DB_HOST, DB_PORT, DB_NAME, DB_USER e DB_PASSWORD.

Os principais nomes de variáveis configurados incluem APP_URL, PORT, TRUST_PROXY_HOPS, AUTH_SERVICE_URL, LOG_SERVICE_URL, JWT_SECRET, SESSION_SECRET, INTERNAL_SERVICE_SECRET, variáveis DB_*, MAIL_*, MINIO_*, STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID, GROQ_API_KEY e GROQ_MODEL. Os valores são mantidos no ambiente de implantação e não são reproduzidos neste relatório. A implantação utiliza imagens versionadas/tag latest no Docker Hub e atualização da Stack pelo Portainer.

3.3 Fluxo de uma requisição

Exemplo: um usuário autenticado consulta seus projetos. Primeiro, o navegador envia uma requisição HTTPS ao domínio público. A aplicação Express recebe a chamada e extrai o token Bearer da sessão. O backend consulta o auth-service pela rede interna, enviando o token e o segredo de comunicação entre serviços. O auth-service valida a sessão e devolve a identidade e as permissões do usuário. A aplicação consulta o MariaDB com filtros de propriedade associados ao usuário autenticado. A operação relevante pode gerar um evento de auditoria, enviado ao log-service e registrado no Redis Streams. Por fim, a aplicação retorna os dados autorizados ao navegador.

Nas operações de criação, atualização, execução ou exclusão, o mesmo princípio é aplicado: autenticação, autorização, validação dos dados, verificação de propriedade, persistência e registro de evento. A interface não é considerada uma barreira de segurança; as regras são verificadas no backend.

3.4 Fluxo de assinatura

O usuário autenticado seleciona o plano Pro e solicita a criação de uma sessão Stripe Checkout. O backend valida a sessão e as configurações, cria ou reutiliza o cliente Stripe e solicita um Checkout com mode=subscription, associando o identificador do usuário aos metadados da sessão e da assinatura. O usuário conclui o pagamento no ambiente de teste do Stripe. O Stripe envia o evento checkout.session.completed ao endpoint /api/billing/webhook.

O webhook valida a assinatura HMAC do cabeçalho stripe-signature e a janela temporal do timestamp, rejeitando eventos inválidos ou fora da tolerância configurada. Para checkout.session.completed, o backend consulta a assinatura no Stripe e persiste os identificadores de cliente, assinatura e preço, o plano, o status e o período de cobrança na tabela assinaturas. Eventos de atualização/cancelamento de assinatura e falha de pagamento também são tratados para refletir o estado no banco.

A tabela stripe_eventos registra o identificador externo do evento com restrição UNIQUE, evitando que o mesmo evento seja processado mais de uma vez. A implementação foi ajustada para usar a restrição única na deduplicação, removendo uma consulta SELECT ... FOR UPDATE que havia contribuído para deadlock em entregas concorrentes. Após o deploy da correção, o fluxo de teste foi concluído e a conta passou a exibir o plano Pro ativo. O cliente pode abrir o Stripe Billing Portal pela rota /api/billing/portal para gerenciar a assinatura; alterações posteriores recebidas por webhook atualizam o estado persistido.

Os limites são aplicados na API, e não apenas pela interface. No plano Free, o produto limita a quantidade de projetos a dois e as execuções HTTP a três por mês; o modo de simulação sem URL é exclusivo do Pro. O

plano Pro libera projetos e execuções sem limite mensal definido pelo produto e permite o uso da simulação. O status da assinatura é consultado pelo endpoint /api/billing/status.

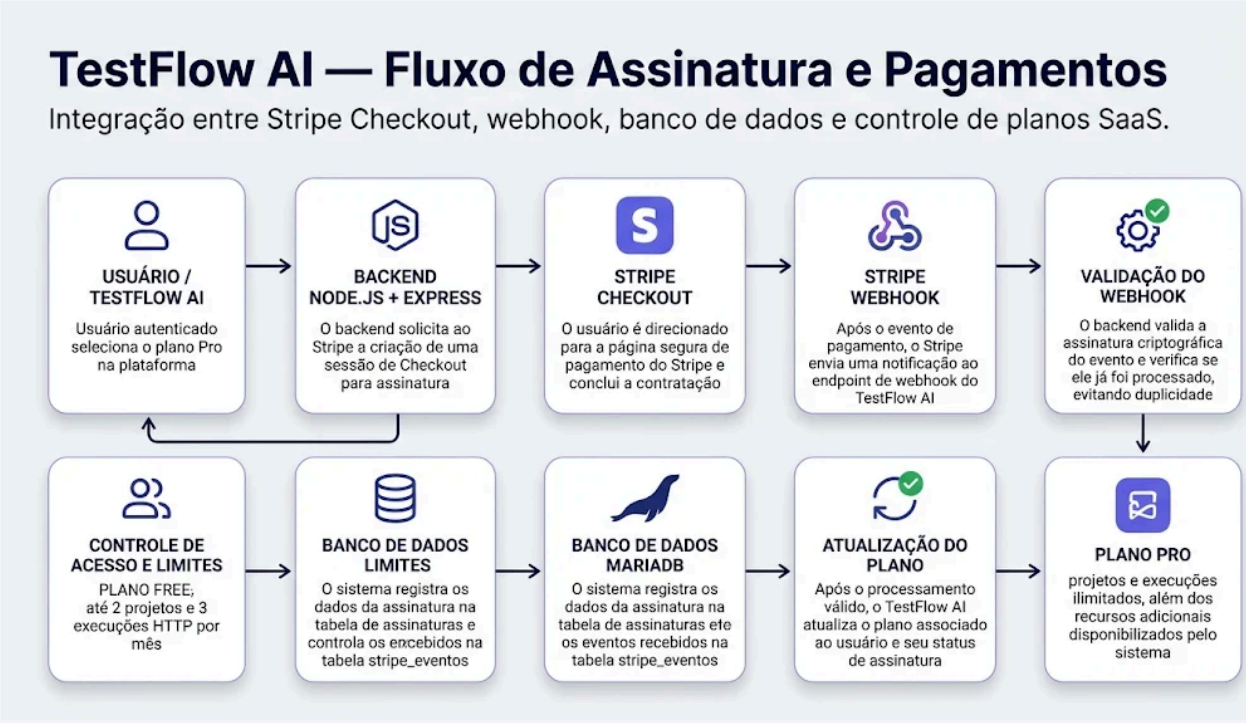


Figura 2 — Fluxo Checkout → webhook validado → banco → limites Free/Pro. *Fonte: Elaborado pela autora (2026).*

3.5 Modelo de dados do domínio

O modelo relacional utiliza as tabelas de domínio e de cobrança. usuarios representa as contas; projetos registra os projetos de teste associados ao proprietário; casos_teste armazena os cenários HTTP ou de simulação; execucoes registra resultados e duração; assinaturas mantém a relação entre usuário e cobrança Stripe; e stripe_eventos registra os eventos externos já recebidos. A estrutura evidencias foi prevista para metadados de arquivos, mas o upload e a recuperação de objetos ainda não estão integrados.

A migração migration-v9-saas.sql acrescenta os campos de modo de execução e dados JSON dos cenários, cria assinaturas com chave estrangeira para usuarios e índices únicos para usuário, cliente Stripe e assinatura Stripe. Também cria stripe_eventos com chave única para stripe_event_id. A migração de domínio v8 deve anteceder a v9. As chaves estrangeiras protegem a integridade referencial, enquanto as consultas do backend aplicam o isolamento por usuario_id.

Tabela	Finalidade	Relacionamentos / observações
usuarios	Contas de acesso	Entidade proprietária dos dados
projetos	Projetos de teste	Associados ao usuario_id
casos_teste	Cenários HTTP e simulação	Associados a projeto e proprietário
execucoes	Resultados e histórico	Associadas a casos/projetos/usuário
assinaturas	Plano e estado de cobrança	FK usuario_id → usuarios.id; UNIQUE por usuário/cliente/assinatura
stripe_eventos	Idempotência de eventos Stripe	UNIQUE stripe_event_id
evidencias	Metadados previstos de arquivos	Upload MinIO ainda não integrado

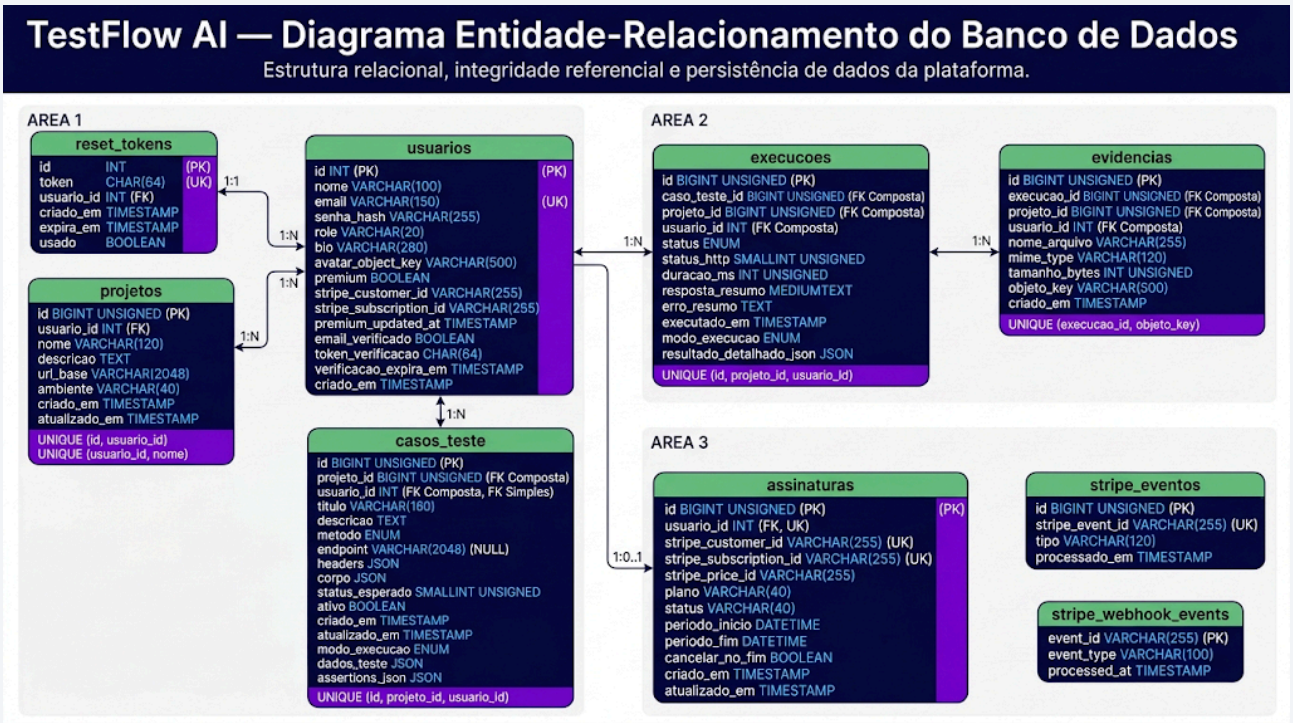


Figura 3 — Diagrama entidade-relacionamento ou esquema das tabelas e chaves. *Fonte: Elaborado pela autora (2026).*

3.6 Segurança

A autenticação utiliza tokens JWT validados pelo auth-service; as senhas são armazenadas com hash bcryptjs. A autorização utiliza papéis e permissões RBAC, incluindo área administrativa restrita. A API administrativa também exige permissão específica, não sendo suficiente ocultar os links na interface. As operações de domínio verificam o usuário proprietário para impedir que uma conta consulte ou altere projetos, casos e execuções de outra conta.

As consultas ao MariaDB são parametrizadas e a estrutura relacional utiliza chaves estrangeiras. O backend aplica Helmet, política de segurança de conteúdo, rate limiting e respostas genéricas para erros internos. A comunicação entre serviços internos utiliza segredo compartilhado por variável de ambiente. Chaves de API, senhas, tokens e segredos são configurados no ambiente da Stack e não devem ser incluídos no repositório ou nas capturas de tela.

O executor HTTP recebe apenas endpoints relativos ao domínio base cadastrado, valida DNS e bloqueia destinos locais, privados ou não permitidos, não segue redirecionamentos, aplica timeout e limita o tamanho da resposta armazenada. Essas restrições reduzem riscos de SSRF e consumo excessivo de recursos. O teste de APIs internas/privadas não é permitido nesta versão.

Ação	Admin	Usuário autenticado	Visitante
Criar e gerenciar os próprios projetos/casos	Conforme permissões	Sim	Não
Consultar dados de outro usuário	Somente funções administrativas autorizadas	Não	Não
Acessar painel administrativo	Sim, com role e permissão	Não	Não
Consultar plano e cobrança próprios	Sim	Sim	Não
Executar simulação sem URL	Se Pro ativo	Somente Pro ativo	Não

3.7 Observabilidade, CI/CD e documentação da API

A observabilidade básica é composta por health checks definidos no Docker Compose para app, auth-service, log-service e Redis, além de eventos de auditoria enviados ao log-service e Redis Streams. O endpoint /health da aplicação permite verificar a saúde básica do serviço. Os logs do backend são utilizados para diagnóstico de falhas, sem expor intencionalmente credenciais.

O workflow .github/workflows/ci-cd.yml executa testes do backend, build do frontend, construção das imagens, subida da composição em ambiente de CI e verificações de saúde da aplicação e dos microsserviços. Após a aprovação do job de testes na branch main, o workflow pode publicar as imagens no Docker Hub quando os secrets necessários estão configurados e pode acionar o webhook de implantação do Portainer. A documentação técnica do projeto está em DOCUMENTACAO.md e é referenciada pelo repositório.

Não foi identificada evidência de uma interface Swagger/OpenAPI publicada e validada; por isso, a documentação da API é descrita neste relatório pelas rotas principais, sem afirmar que existe portal Swagger operacional.

3.8 Camadas IaaS, PaaS e SaaS do projeto

No contexto do projeto, a camada IaaS corresponde à infraestrutura computacional, rede, armazenamento e máquina virtual disponibilizados para a execução da Stack, cuja administração depende do ambiente da disciplina e do provedor. A camada PaaS é representada pelos serviços e ferramentas gerenciados sobre essa infraestrutura, como Docker, Portainer, MariaDB, Redis e MinIO, cuja configuração e integração são responsabilidade do desenvolvimento e da operação do projeto. A camada SaaS é o próprio TestFlow AI, disponibilizado ao usuário final pelo navegador, além de serviços externos consumidos como Stripe, GitHub e Groq.

Camada	Aplicação no projeto	Responsabilidade de gestão
IaaS	VM, rede, disco e infraestrutura base da disciplina	Provedor/ambiente da disciplina; administração conforme acesso concedido
PaaS	Docker, Portainer, MariaDB, Redis e MinIO	Configuração e operação divididas entre infraestrutura da disciplina e desenvolvimento
SaaS	TestFlow AI; Stripe, GitHub e Groq como serviços externos consumidos	Produto: desenvolvimento/operação; terceiros: seus respectivos provedores

4 Quadro de entregas

Nº	Atividade	Planejada	Realizada (BRT)	Situação
7	Plano Premium — Stripe	09/10/2026	16/10/2026 20:50	Entregue com atraso
8	SaaS, PaaS, IaaS e documentação	16/10/2026	23/10/2026 19:38	Entregue com atraso
9	Fork e plano de negócio	23/10/2026	26/10/2026 12:16	Entregue com atraso
10	Produto publicado no domínio	06/11/2026	09/11/2026 21:41	Entregue com atraso
11	SaaS completo — planos e Stripe	13/11/2026	12/11/2026 22:12 (PR #16)	Entregue no prazo

Observação sobre datas: para a atividade 11, o registro de entrega considerado é o PR #16, aberto antes do prazo final. Correções finais de webhook e confirmação visual foram realizadas em 13/11/2026. As datas de atividades 7 e 8 foram convertidas para BRT a partir do histórico do GitHub.

5 Atividades realizadas

As fichas a seguir descrevem o escopo realizado, a data registrada e as dificuldades encontradas em cada atividade. As evidências visuais correspondentes foram inseridas ao longo das respectivas seções,

complementando os links para o repositório, commits, issues e pull requests utilizados para documentar e comprovar as entregas.

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Data planejada: 09/10/2026. Data registrada: 16/10/2026, 20h50 (BRT), commit 47a42c6. Situação: entregue com atraso.

A atividade implementou o primeiro fluxo de plano pago em ambiente de teste, conectando o produto ao Stripe Checkout e ao webhook para reconhecer o estado Premium. O trabalho incluiu a associação do usuário à cobrança e a atualização do plano a partir dos dados recebidos do provedor. O histórico também registra documentação específica da atividade no commit 18613dd, de 16/10/2026, às 21h41 (BRT).

Dificuldades e resolução: a integração exige que o backend valide a origem e a autenticidade dos eventos enviados pelo Stripe, além de manter a associação entre cliente, assinatura e usuário. O fluxo foi evoluído para utilizar eventos assinados e persistência no banco, preparando a estrutura que seria ampliada na atividade 11.

Evidência principal: https://github.com/joicesoato/testflow-ai/commit/47a42c605fa2bda93c5630f959126a3cd4afc9c0

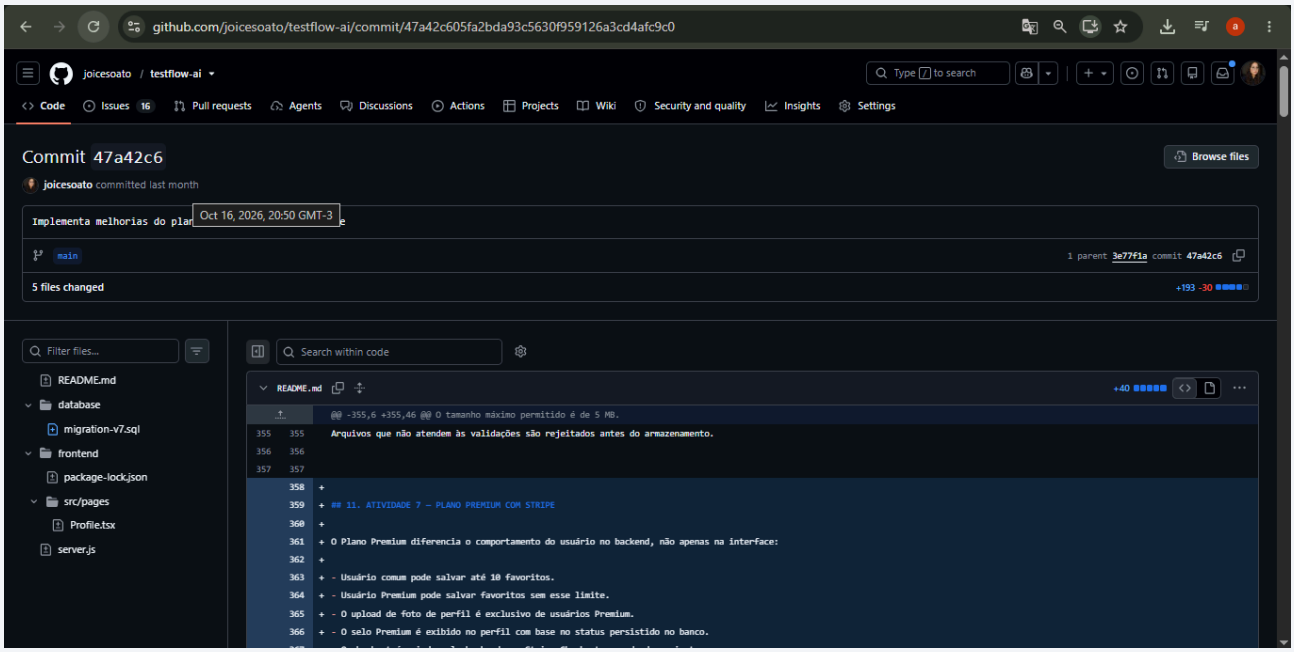


Figura 4 — Evidência da entrega: commit/registro com data, hora e link visíveis.

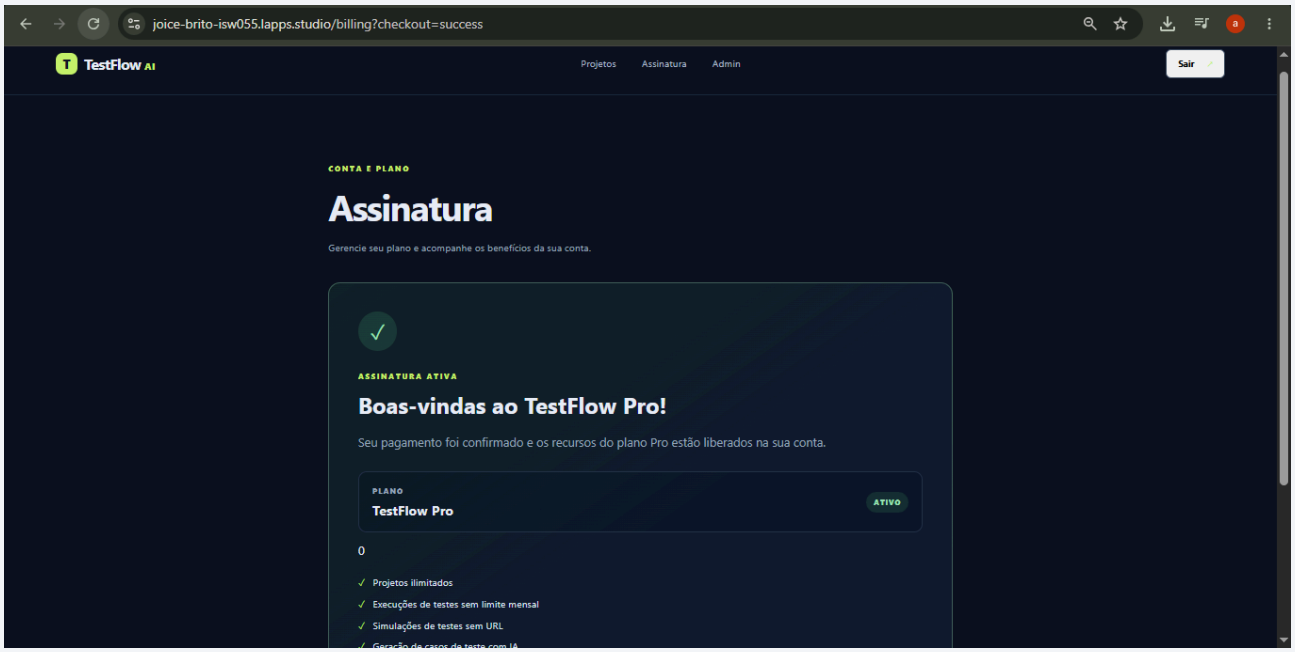


Figura 5 — Resultado: Checkout de teste e plano Premium/Pro reconhecido.

5.2 Atividade 8 — SaaS, PaaS, IaaS e documentação

Data planejada: 16/10/2026. Data registrada: 23/10/2026, 19h38 (BRT), commit c334d09. Situação: entregue com atraso.

A atividade consistiu em identificar as camadas de serviço utilizadas pelo produto e realizar documentação reversa da solução. Foram descritos os componentes, as responsabilidades, as tecnologias, os fluxos principais, as regras de negócio e o modelo de dados, distinguindo o que já estava implementado do que ainda era uma evolução planejada. A documentação técnica foi consolidada em DOCUMENTACAO.md e posteriormente complementada para refletir a arquitetura TestFlow AI e o estado real do repositório.

Dificuldades e resolução: a principal dificuldade foi separar infraestrutura provisionada de funcionalidade efetivamente integrada. Por exemplo, o MinIO está presente na Stack, mas o upload de evidências não está concluído. Essa distinção foi registrada na documentação para evitar apresentar como pronto um recurso que ainda exige desenvolvimento.

Evidência principal: https://github.com/joicesoato/testflow-ai/commit/c334d0946b82564d1083ffbea062e9981a631bcc

Documentação atual: https://github.com/joicesoato/testflow-ai/blob/main/DOCUMENTACAO.md

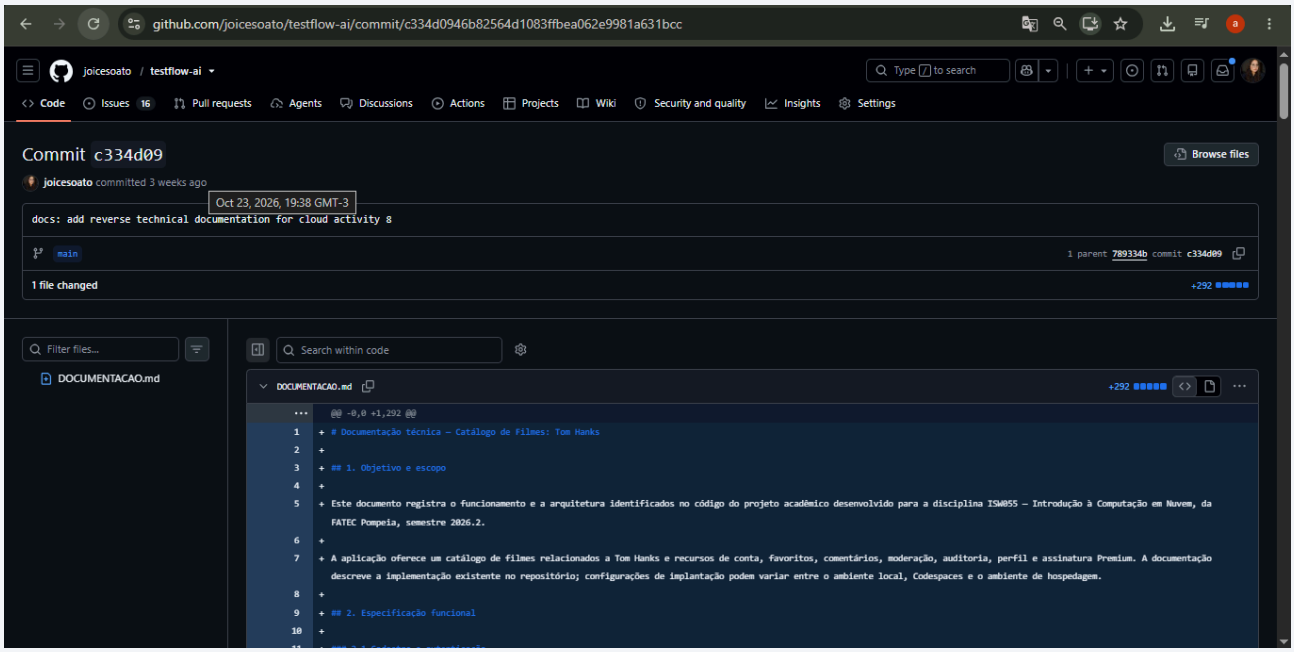


Figura 6 — Evidência da entrega: commit e arquivo DOCUMENTACAO.md no GitHub.



Figura 7 — Resultado: documentação/arquitetura ou aplicação documentada.

5.3 Atividade 9 — Fork e plano de negócio

Data planejada: 23/10/2026. Data registrada: 26/10/2026, 12h16 (BRT), issue #15. Situação: entregue com atraso.

O repositório joicesoato/testflow-ai foi criado como novo espaço de evolução do produto, preservando o histórico do projeto de origem. A issue #15 formalizou a entrega ao professor e apontou o novo repositório, o README, o histórico de commits e as issues de planejamento. O plano de negócio descreve o problema, o público-alvo, a proposta de valor, alternativas existentes, modelo de receita, métricas e Lean Canvas. A decisão de produto foi transformar a base de catálogo em uma plataforma SaaS de organização e automação inteligente de testes de APIs REST.

O planejamento foi organizado em issues para landing page, gestão de projetos e casos, executor seguro, histórico, publicação, planos Free/Pro, Stripe, painel administrativo, documentação e CI/CD. O reaproveitamento da arquitetura reduziu o trabalho de infraestrutura e permitiu concentrar a evolução nas funcionalidades do novo domínio.

Dificuldades e resolução: foi necessário preservar o histórico Git e separar claramente a identidade do novo produto da aplicação anterior. A criação do novo repositório e a atualização do README e do backlog estabeleceram a base para as atividades de implementação e publicação.

Evidência principal: https://github.com/joicesoato/testflow-ai/issues/15

README: https://github.com/joicesoato/testflow-ai/blob/main/README.md

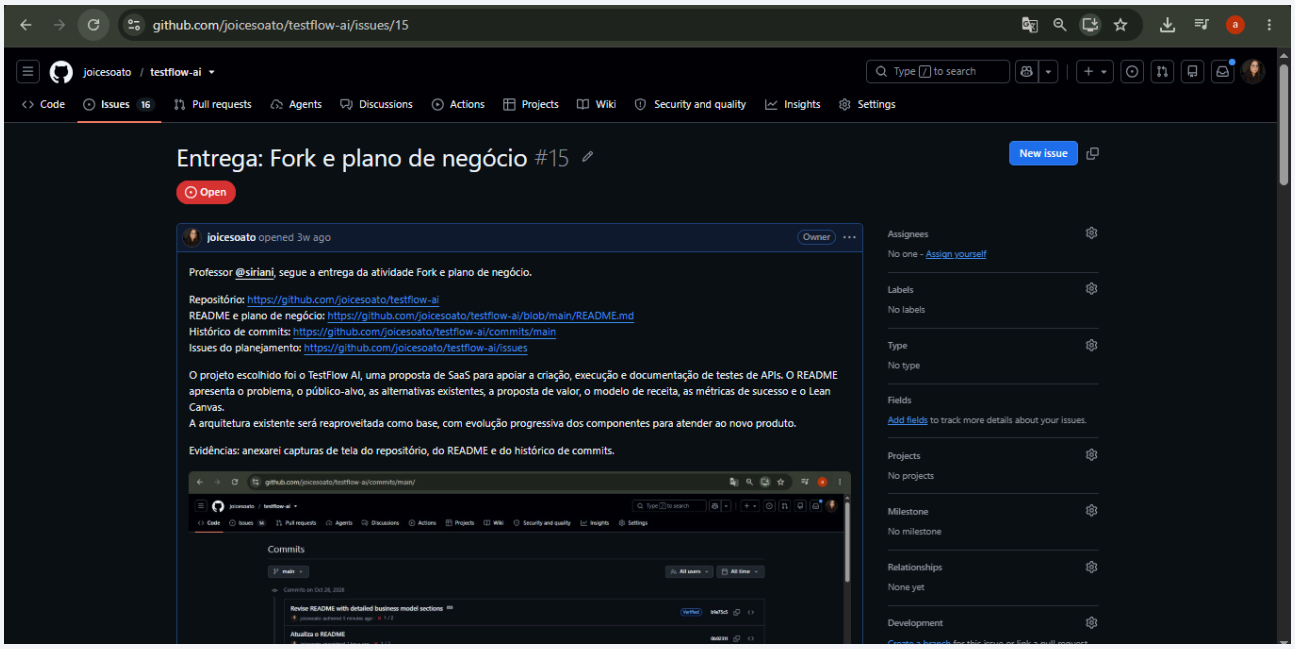


Figura 8 — Evidência da entrega: issue #15, repositório, data e histórico.

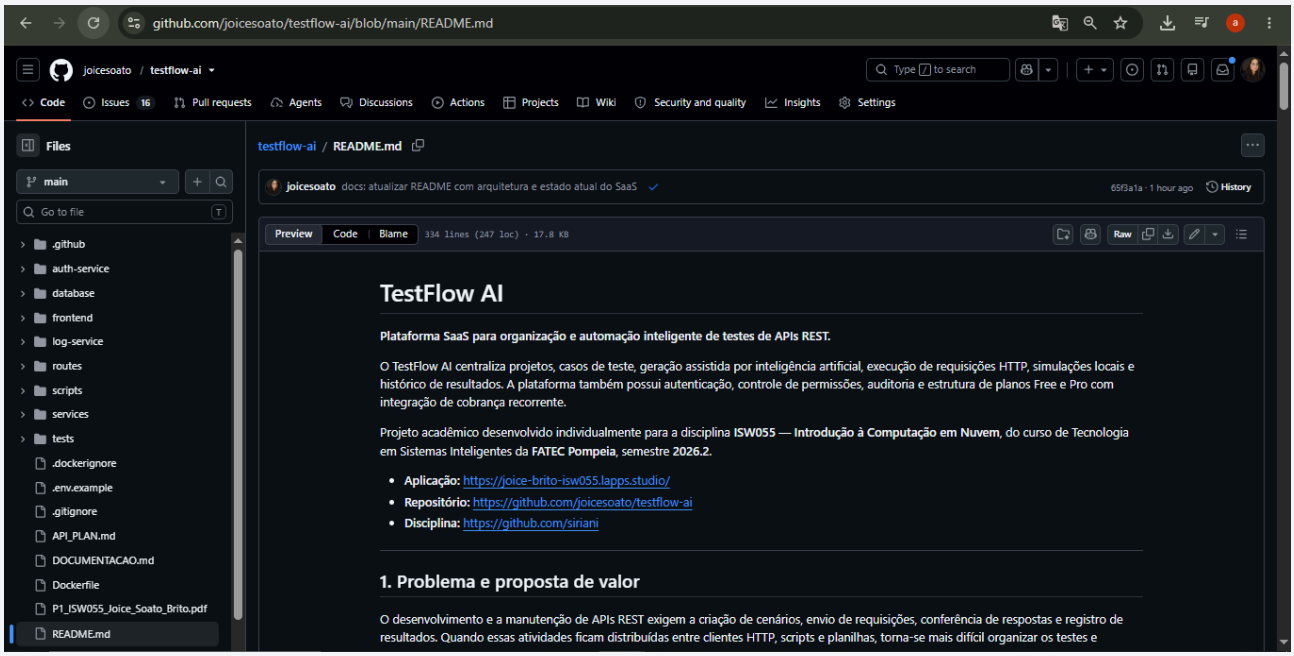


Figura 9 — Resultado: README com proposta de valor, modelo de negócio e Lean Canvas.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Data planejada: 06/11/2026. Data registrada: 09/11/2026, 21h41 (BRT), issue de entrega #17; commit principal de implantação 12955b6 em 09/11/2026, 21h40 (BRT). Situação: entregue com atraso.

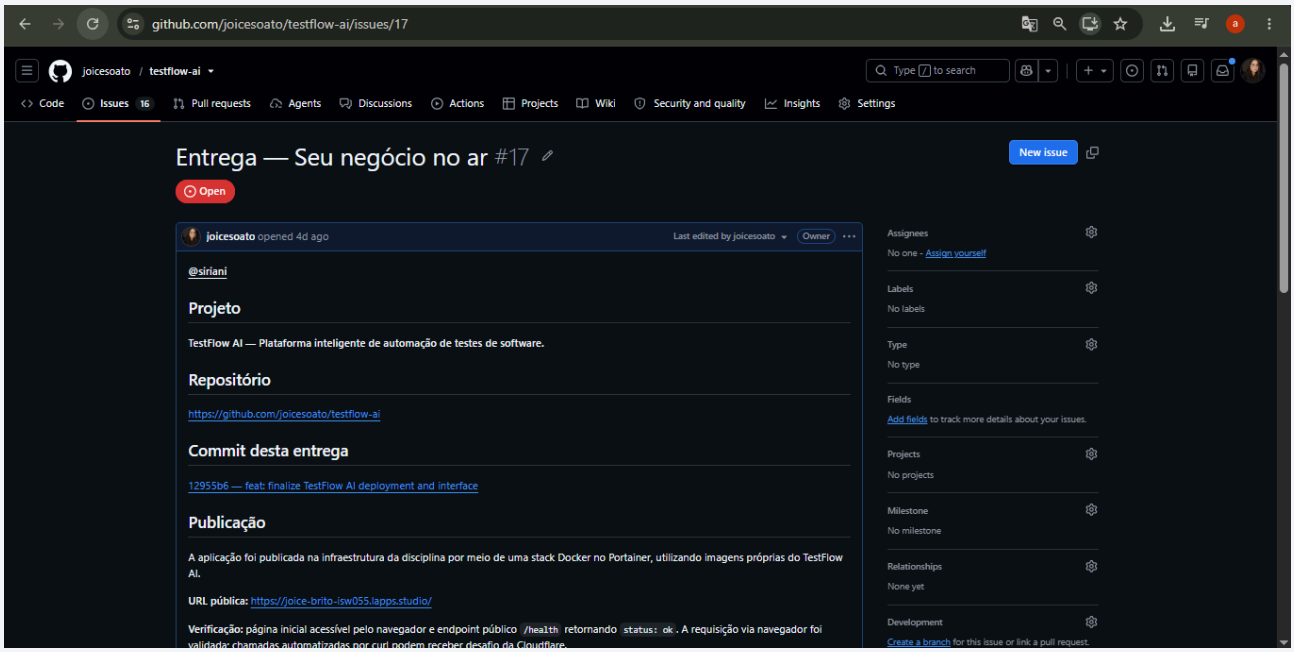
A aplicação TestFlow AI foi adaptada e publicada na infraestrutura da disciplina, utilizando a Stack Docker gerenciada pelo Portainer e imagens próprias no Docker Hub. A entrega documentou a URL pública, o endpoint /health e a organização dos serviços app, auth-service, log-service, Redis e MinIO. O sistema passou a apresentar a identidade do novo produto e as telas voltadas à gestão de projetos e casos de teste. O README e a issue de entrega registram a publicação e os itens de validação.

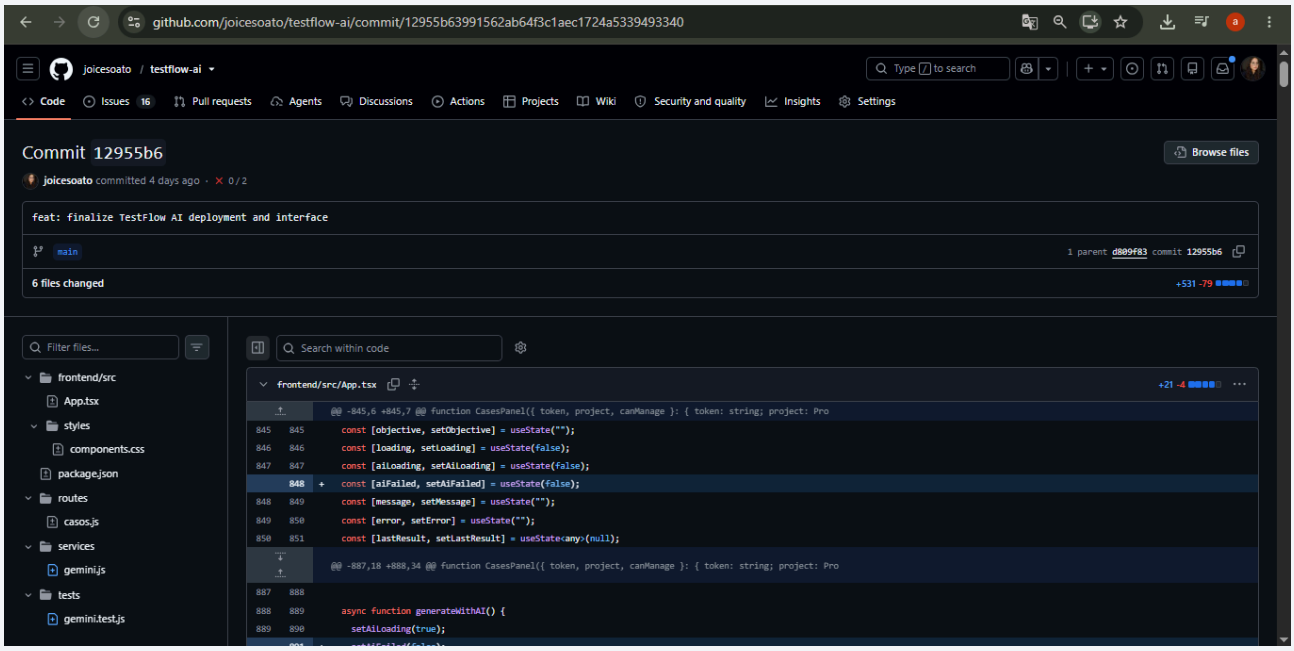
A issue #17 também diferencia o que foi verificado do que ainda precisava de evidência adicional: página inicial e endpoint de saúde foram verificados; as capturas de listagem/detalhe de projeto, containers, permissões, isolamento, logs e upload deveriam ser anexadas separadamente. O upload de evidências no MinIO permaneceu como limitação conhecida e não é apresentado como funcionalidade concluída.

Dificuldades e resolução: a implantação exigiu adaptar imagens, variáveis de ambiente, portas e comunicação interna dos serviços. Também houve necessidade de diagnosticar o acesso público e a camada de proxy. A aplicação foi publicada no domínio da disciplina, com health check configurado para facilitar a verificação operacional.

Evidência principal: https://github.com/joicesoato/testflow-ai/issues/17

Commit de implantação: https://github.com/joicesoato/testflow-ai/commit/12955b63991562ab64f3c1aec1724a5339493340


github.com/joicesoato/testflow-ai/issues/17
joicesoato / testflow-ai
Code
Issues 16
Pull requests
Agents
Discussions
Actions
Projects
Wiki
Security and quality
Insights
Settings
Entrega — Seu negócio no ar #17
Open
New issue
joicesoato opened 4d ago
Last edited by joicesoato
Owner
@siriani
Projeto
TestFlow AI — Plataforma inteligente de automação de testes de software.
Repositório
https://github.com/joicesoato/testflow-ai
Commit desta entrega
12955b6 — feat: finalize TestFlow AI deployment and interface
Publicação
A aplicação foi publicada na infraestrutura da disciplina por meio de uma stack Docker no Portainer, utilizando imagens próprias do TestFlow AI.
URL pública: https://joice-brito-isw055.iapps.studio/
Verificação: página inicial acessível pelo navegador e endpoint público /health retornando status: ok. A requisição via navegador foi
Assignees
No one - Assign yourself
Labels
No labels
Type
No type
Fields
Add fields to track more details about your issues.
Projects
No projects
Milestone
No milestone
Relationships
None yet
Development
Create a branch for this issue or link a pull request.


github.com/joicesoato/testflow-ai/commit/12955b63991562ab64f3c1aec1724a5339493340
joicesoato / testflow-ai
Code
Issues 16
Pull requests
Agents
Discussions
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commit 12955b6
Browse files
joicesoato committed 4 days ago
feat: finalize TestFlow AI deployment and interface
main
1 parent d809f83 commit 12955b6
6 files changed
+531 -79
Filter files...
frontend/src
App.tsx
styles
components.css
package.json
routes
casos.js
services
gemini.js
tests
gemini.test.js
Search within code
frontend/src/App.tsx
+21 -4
@@ -845,6 +845,7 @@ function CasesPanel({ token, project, canManage }: { token: string; project: Pro
const [objective, setObjective] = useState("");
const [loading, setLoading] = useState(false);
const [aiLoading, setAiLoading] = useState(false);
+ const [aiFailed, setAiFailed] = useState(false);
const [message, setMessage] = useState("");
const [error, setError] = useState("");
const [lastResult, setLastResult] = useState<any>(null);
@@ -887,18 +888,34 @@ function CasesPanel({ token, project, canManage }: { token: string; project: Pro
async function generateWithAI() {
setAiLoading(true);

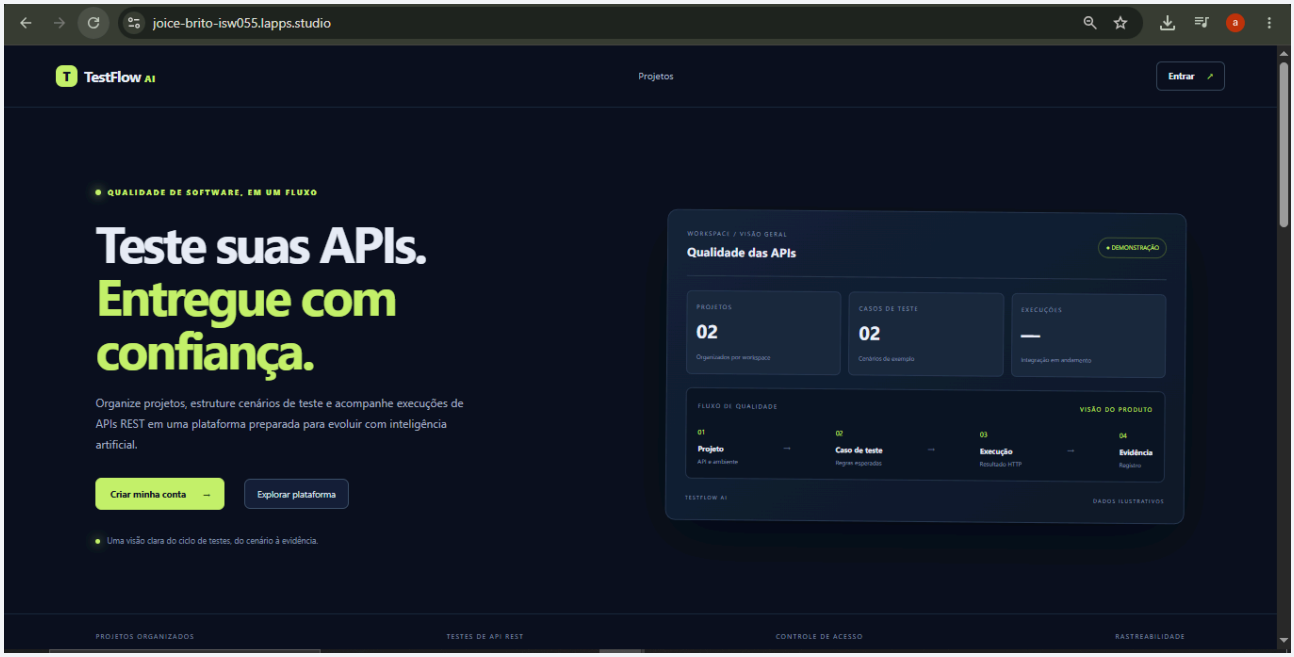


Figura 10 — Evidência da entrega: issue #17, commit e URL pública.

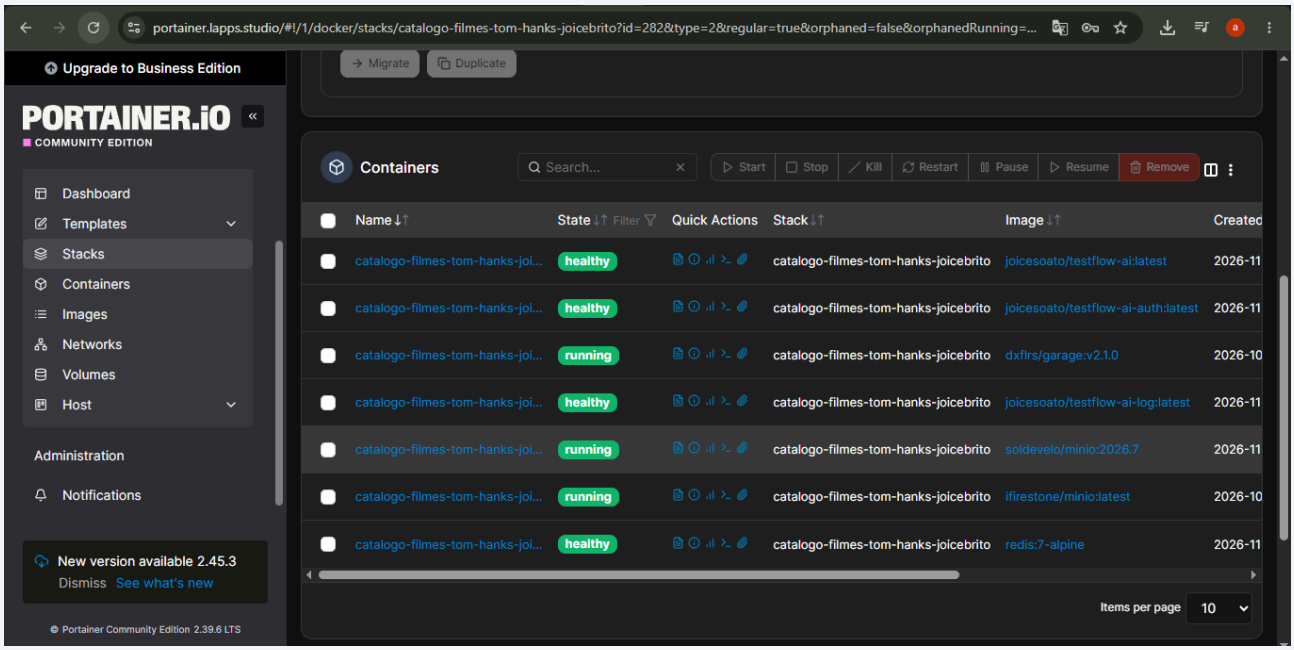


Figura 11 — Resultado: landing page, tela de projetos ou Portainer com serviços ativos.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Data planejada: 13/11/2026. Data de entrega registrada: 12/11/2026, 22h12 (BRT), PR #16. Situação: entregue dentro do prazo, com correções finais em 13/11/2026.

A atividade consolidou os planos Free e Pro, os limites aplicados no backend, o modo de simulação sem URL e o fluxo de assinatura recorrente. O PR #16 implementou a simulação declarativa com dados JSON,

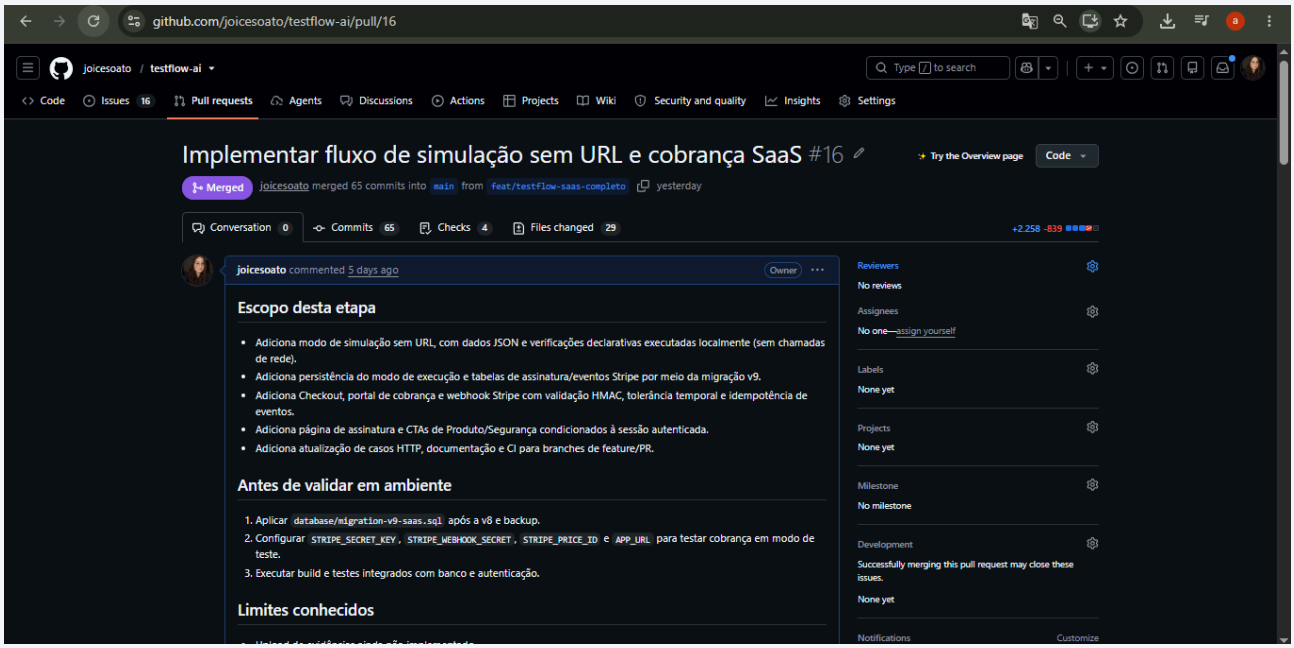
persistência do modo de execução, Checkout, Billing Portal e webhook Stripe com validação HMAC, tolerância temporal e idempotência. A API aplica os limites de uso: Free com até dois projetos e três execuções HTTP por mês; Pro com projetos e execuções sem limite mensal definido pelo plano e acesso à simulação sem URL.

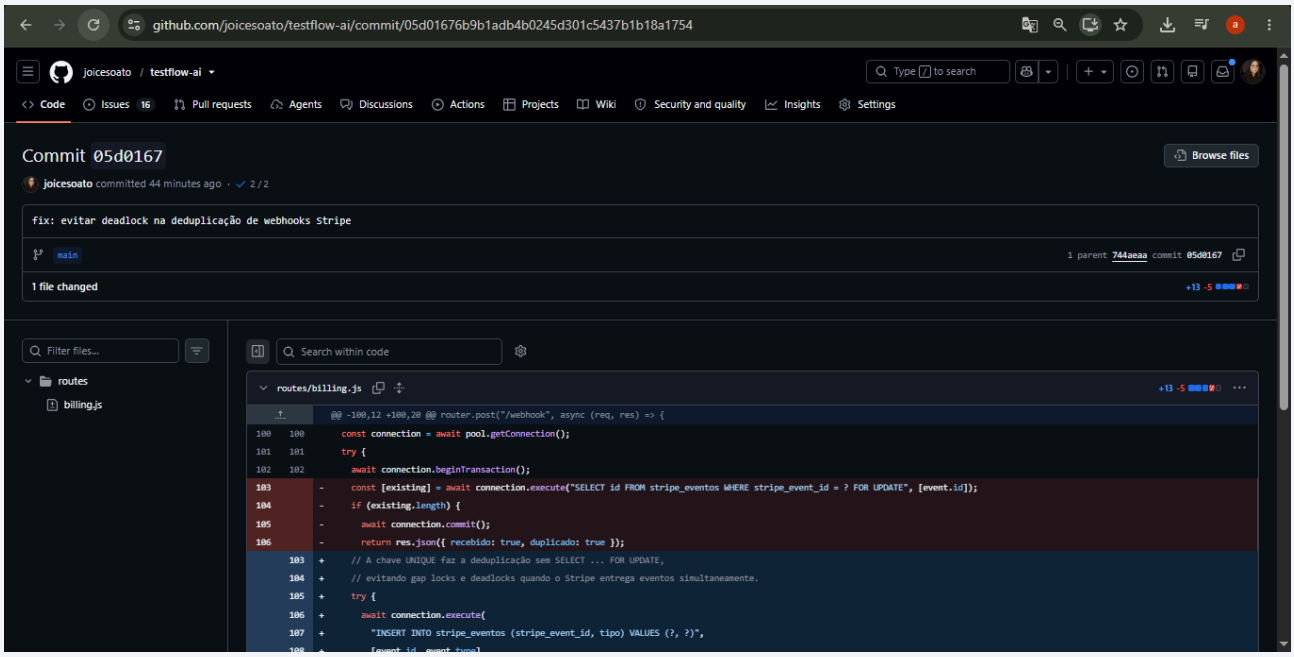
Na validação integrada, o Checkout foi realizado com cartão de teste do Stripe. Inicialmente, a entrega do webhook apresentou falha por deadlock na deduplicação de eventos, embora o pagamento de teste tivesse sido concluído. A correção substituiu a consulta de bloqueio por inserção protegida pela chave única de stripe_eventos, com tratamento de evento duplicado. Também foi adicionada atualização periódica do status na tela de retorno do Checkout. Após publicar a imagem corrigida e atualizar a Stack do Portainer, o evento foi processado e a conta passou a exibir o plano Pro ativo. A validação foi feita em modo de teste, não representando cobrança financeira real.

O repositório também registra a integração de geração de casos com Groq e refinamentos de interface e administração em pull request separado. O escopo final do relatório considera apenas funcionalidades integradas à branch principal e comprovadas no ambiente; o PR #18, que aparece como rascunho e sem merge na consulta realizada, não é usado como prova de implantação dessas alterações.

Dificuldades e resolução: a maior dificuldade foi garantir que o estado exibido na interface acompanhasse a confirmação assíncrona do Stripe. O diagnóstico dos logs identificou deadlock no processamento do webhook. A deduplicação foi corrigida com a restrição UNIQUE e o fluxo foi novamente implantado. A ativação efetiva do plano Pro após a atualização confirmou o funcionamento do caminho Checkout → webhook → banco → consulta de plano.

Evidências principais: https://github.com/joicesoato/testflow-ai/pull/16 · https://github.com/joicesoato/testflow-ai/commit/05d01676b9b1adb4b0245d301c5437b1b18a1754


github.com/joicesoato/testflow-ai/pull/16
joicesoato / testflow-ai
Code
Issues 16
Pull requests
Agents
Discussions
Actions
Projects
Wiki
Security and quality
Insights
Settings
Implementar fluxo de simulação sem URL e cobrança SaaS #16
Merged
joicesoato merged 65 commits into main from feat/testflow-saas-completo yesterday
Conversation 0
Commits 65
Checks 4
Files changed 29
+2.258 -839
joicesoato commented 5 days ago
Owner
Escopo desta etapa
Adiciona modo de simulação sem URL, com dados JSON e verificações declarativas executadas localmente (sem chamadas de rede).
Adiciona persistência do modo de execução e tabelas de assinatura/eventos Stripe por meio da migração v9.
Adiciona Checkout, portal de cobrança e webhook Stripe com validação HMAC, tolerância temporal e idempotência de eventos.
Adiciona página de assinatura e CTAs de Produto/Segurança condicionados à sessão autenticada.
Adiciona atualização de casos HTTP, documentação e CI para branches de feature/PR.
Antes de validar em ambiente
1. Aplicar database/migration-v9-saas.sql após a v8 e backup.
2. Configurar STRIPE_SECRET_KEY, STRIPE_WEBHOOK_SECRET, STRIPE_PRICE_ID e APP_URL para testar cobrança em modo de teste.
3. Executar build e testes integrados com banco e autenticação.
Limites conhecidos
Reviewers
No reviews
Assignees
No one—assign yourself
Labels
None yet
Projects
None yet
Milestone
No milestone
Development
Successfully merging this pull request may close these issues.
None yet
Notifications
Customize


github.com/joicesoato/testflow-ai/commit/05d01676b9b1adb4b0245d301c5437b1b18a1754
joicesoato / testflow-ai
Code
Issues 16
Pull requests
Agents
Discussions
Actions
Projects
Wiki
Security and quality
Insights
Settings
Commit 05d0167
Browse files
joicesoato committed 44 minutes ago · 2 / 2
fix: evitar deadlock na deduplicação de webhooks Stripe
main
1 parent 744aeaa commit 05d0167
1 file changed
+13 -5
Filter files...
routes
billing.js
Search within code
routes/billing.js
+13 -5
@@ -100,12 +100,20 @@ router.post("/webhook", async (req, res) => {
const connection = await pool.getConnection();
try {
await connection.beginTransaction();
- const [existing] = await connection.execute("SELECT id FROM stripe_eventos WHERE stripe_event_id = ? FOR UPDATE", [event.id]);
- if (existing.length) {
- await connection.commit();
- return res.json({ recebido: true, duplicado: true });
- }
+ // A chave UNIQUE faz a deduplicação sem SELECT ... FOR UPDATE,
+ // evitando gap locks e deadlocks quando o Stripe entrega eventos simultaneamente.
+ try {
+ await connection.execute(
+ "INSERT INTO stripe_eventos (stripe_event_id, tipo) VALUES (?, ?)",

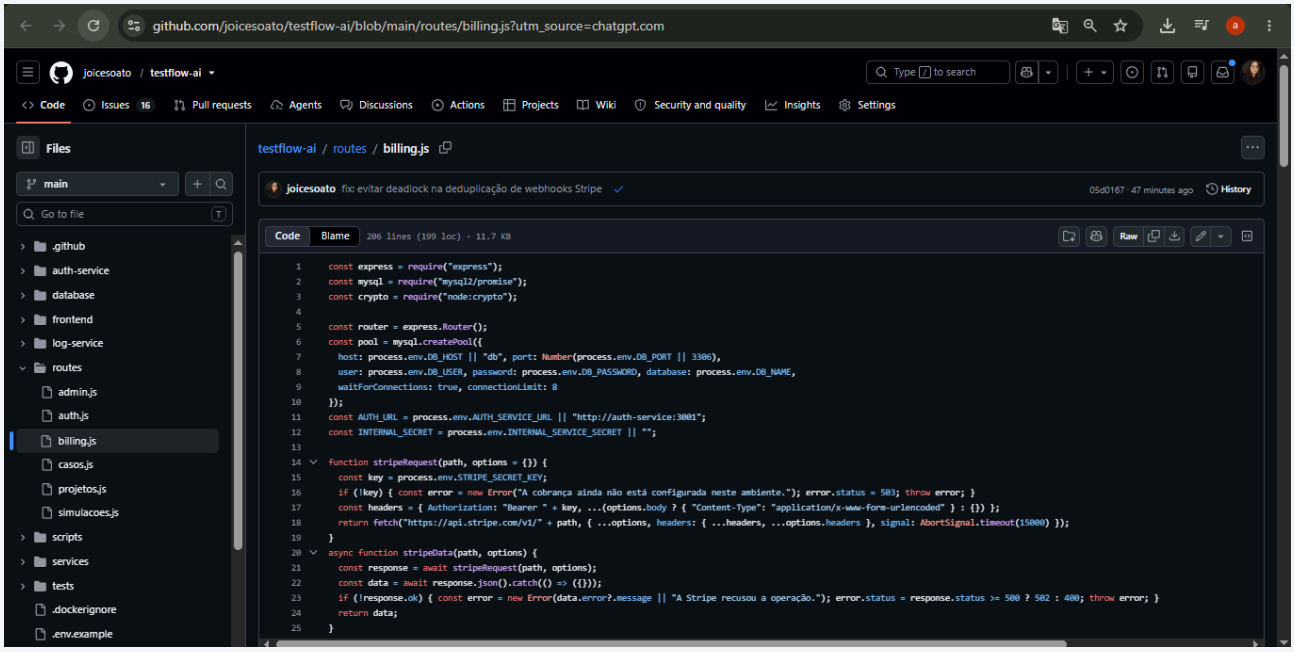


Figura 12 — Evidência da entrega: PR #16/commits e configuração de cobrança sem expor segredos.

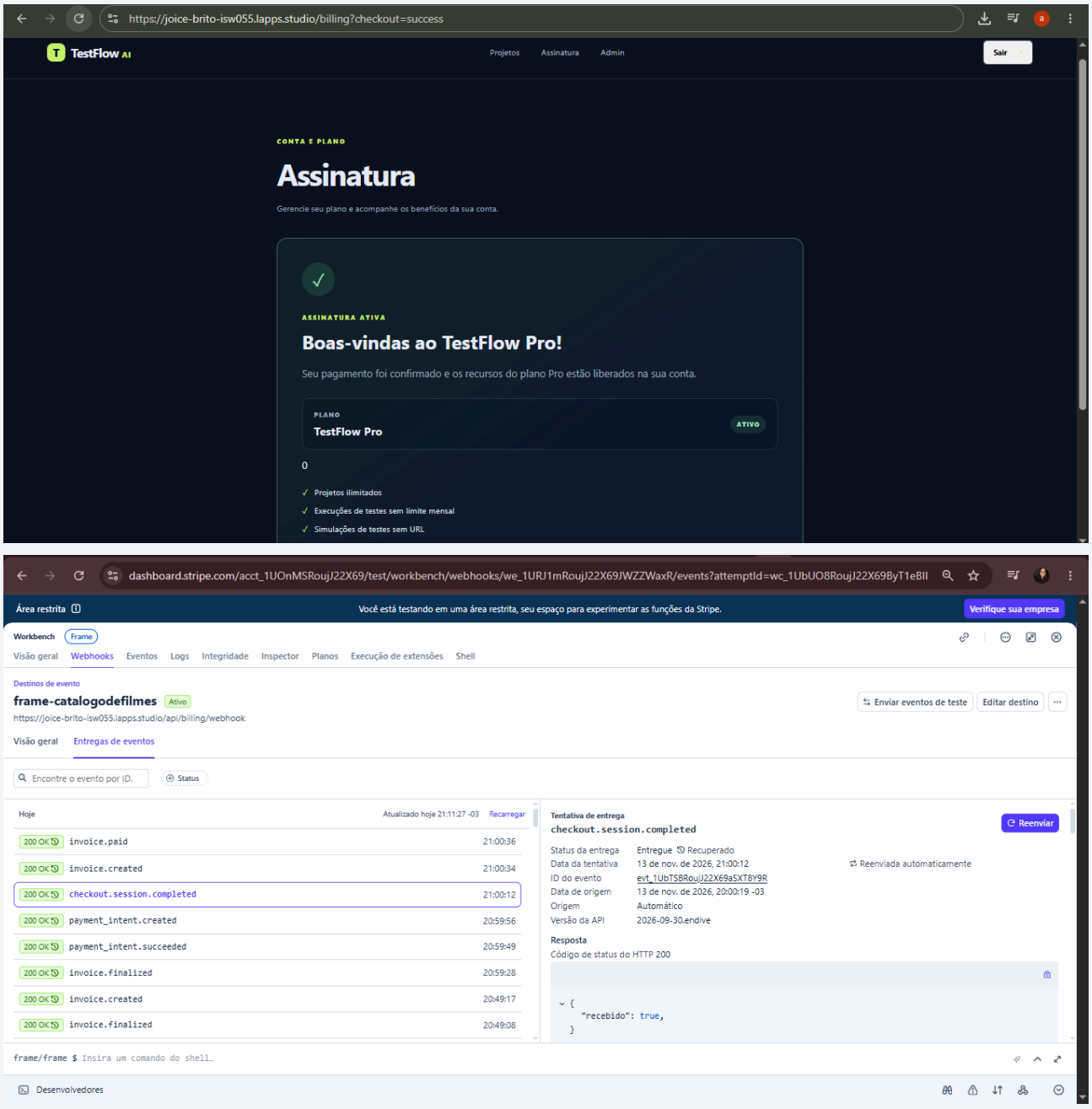


Figura 13 — Resultado: conta exibindo plano Pro ativo e Stripe em modo de teste/webhook concluído.

6 Considerações finais

O segundo bimestre permitiu transformar uma base técnica já existente em um produto com domínio, público-alvo e modelo de receita definidos. A experiência envolveu não apenas a implementação de telas e rotas, mas também decisões de arquitetura, organização de backlog, persistência relacional, comunicação entre microsserviços, implantação conteinerizada, segurança, integração com serviços externos e documentação técnica.

Os pontos de maior complexidade foram a adaptação da infraestrutura ao novo produto, o isolamento das informações por usuário, as proteções do executor HTTP e a consistência do estado da assinatura diante de eventos assíncronos. O diagnóstico do webhook Stripe demonstrou a importância de observar logs, entender concorrência no banco de dados e validar o fluxo completo após cada atualização da aplicação.

Como próximos passos para aproximar o TestFlow AI de um produto comercial, destacam-se a integração efetiva do MinIO para anexos de evidência, ampliação das assertions dos testes HTTP, expansão dos testes de integração e segurança e fortalecimento do monitoramento. A versão entregue representa uma base funcional de SaaS acadêmico, com limitações explicitadas e possibilidades claras de evolução.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem às entregas de minha autoria, verificáveis nos links informados. As capturas de tela inseridas ao longo do documento complementam os registros de código, histórico, issues e pull requests descritos, demonstrando visualmente as implementações, validações e resultados obtidos durante o desenvolvimento do TestFlow AI.

_____

Joice Soato Brito

Pompeia, 13 de novembro de 2026